

Agentic AI & AI Agents Interview Questions & Answers

Beginner · Intermediate · Advanced · Production Design

Senior AI Engineer Level | 2026 Edition

- 1. Beginner — Foundations of AI Agents (Q1–Q10)
- 2. Intermediate — Building Real Agents (Q11–Q30)
- 3. Advanced — Production Agent Systems (Q31–Q50)
- 4. Agent vs Agentic — Key Distinctions (Q51–Q55)
- 5. Architecture Deep Dives — E-commerce, Enterprise, SRE
- 6. Top 20 Must-Memorise Questions
- 7. What Interviewers Ask After Agent Questions
- 8. Interview Checklists

1. Beginner — Foundations of AI Agents

Q1. What is an AI agent?

A system where an LLM doesn't just generate text but can **reason, decide actions, use tools, observe results, and continue toward a goal**.

Normal chatbot: user asks → model responds once.

Agent: user gives goal → model plans, calls tools, observes, iterates → final answer.

- Search documents
- Call APIs
- Query databases
- Plan steps
- Retry after failure
- Execute multi-step workflows

Q2. What is 'agentic AI'?

Agentic AI refers to systems where the model behaves as a **goal-driven actor** rather than a single prompt-response generator. It includes: planning, tool use, memory/state, multi-step execution, feedback loops, and decision-making based on observations.

Q3. Difference between a normal LLM app and an AI agent?

Normal LLM app — User asks a question → model responds once. Fixed, stateless.

AI Agent — User gives a goal → model may plan, call tool, inspect result, call another tool, generate final answer. Multi-step, stateful.

*The key difference is **multi-step decision-making and action execution**.*

Q4. What are the core components of an AI agent?

1. LLM / reasoning model
2. Tools (APIs, DB, search, calculator, browser, etc.)
3. Memory / state
4. Control loop / orchestration
5. Instructions / system prompt
6. Observation of tool results
7. Final answer or action output

Q5. What is a tool in an agent system?

A function or external capability the agent can call. The LLM decides **when** to use the tool and **with what arguments**.

- Get order status
- Search product catalog
- Run SQL query
- Fetch weather data
- Retrieve support docs
- Send email / create Jira ticket

Q6. What is the role of memory in an AI agent?

Memory helps the agent track information across steps or conversations:

- Current user goal
- Previous tool outputs
- Retrieved documents
- User preferences
- Unfinished tasks
- Retry state

Without memory, the agent behaves like disconnected stateless prompt calls.

Q7. What is a multi-step agent workflow?

When the agent completes a task over several steps. Example — 'Find cheapest in-stock laptop and apply my gold discount':

1. Search catalog
2. Find product price
3. Check discount eligibility
4. Calculate discounted price
5. Return result

Q8. Chain vs Agent — what's the difference?

Chain — Fixed sequence, deterministic, developer decides all steps

Agent — Dynamic, model decides what to do next, may choose tools based on task

Q9. Why are agents useful?

When tasks require: tool usage, dynamic decisions, multiple steps, real-time external data, planning and execution.

- Customer support assistant with order lookup + policy retrieval
- Internal enterprise assistant (Confluence + Jira + DB)
- Shopping assistant
- Coding assistant
- Workflow automation

Q10. What are common agent use cases?

- Customer support: order lookup + policy retrieval + ticket creation
- HR assistant: company docs + leave systems
- Coding assistant: GitHub + docs + tools
- Sales assistant: CRM + email drafting
- Research assistant: web + notes + synthesis
- SRE agent: health checks + SSL renewals + incident response

2. Intermediate — Building Real Agents

Q11. What is the typical loop of an AI agent?

Often called the **Reason** → **Act** → **Observe** loop:

1. Receive user goal
2. Send context + tools to LLM
3. LLM decides: answer directly OR call a tool
4. Execute tool
5. Return tool result to LLM as observation
6. LLM decides next action
7. Repeat until final answer or max iterations

Q12. Tool calling vs agentic behavior — what's the difference?

Tool calling — Model can invoke a function — a building block

Agentic behavior — Model decides when/which tool to call, chains multiple calls, updates plan based on results, continues until goal achieved

Tool calling is a primitive. Agentic behavior is the larger decision-making loop.

Q13. What is ReAct in agents?

ReAct = Reason + Act. Model interleaves reasoning (Thought) with action (Act) and result observation (Observe).

```
Thought: I need to look up the order status.  
Act: call_tool(order_lookup, order_id='ORD-123')  
Observe: {status: 'shipped', eta: '2026-06-28'}  
Thought: I have the data. I can answer now.  
Final Answer: Your order ships June 28.
```

Q14. What is agent state?

Structured information the agent carries during execution:

- Conversation history
- Current task / goal
- Retrieved documents
- Tool outputs
- Retry count
- User metadata
- Current plan
- Approval status

In production, explicit structured state is essential — not just a message list.

Q15. Short-term memory vs long-term memory in agents?

Short-term (Current task/session) — Current messages, tool outputs, current plan, temporary context

Long-term (Across sessions) — User preferences, important facts, historical summaries, learned profile data

Q16. What is planning in an AI agent?

Breaking a goal into smaller steps before or during execution.

Example — 'Summarize this incident, find root cause, draft customer update':

1. Read incident logs
2. Summarize timeline
3. Identify probable root cause
4. Draft status update

Not every agent needs explicit planning. For simple tasks it adds cost/latency with no benefit.

Q17. What is an observation in an agent loop?

The result returned by a tool that the agent uses to decide the next step.

```
# Tool call
call_tool('get_order_status', order_id=123)

# Observation returned to agent
{"status": "shipped", "eta": "2026-06-28"}

# Agent uses this in next reasoning step
```

Q18. What is a router agent?

Decides which path/tool/sub-agent should handle the request:

- Billing query → billing workflow
- Refund request → order tool + policy retrieval
- Product question → catalog search
- Technical issue → troubleshooting workflow

Q19. What is a supervisor agent?

Coordinates multiple specialized sub-agents. Decides who does what and combines results.

```
# Example supervisor pattern
Supervisor
  ■■■ retrieval_agent    (searches docs)
  ■■■ sql_agent          (queries DB)
  ■■■ summarizer_agent   (condenses results)
  ■■■ writer_agent       (drafts final response)
```

Q20. What is a multi-agent system?

Multiple specialized agents collaborating on a task.

Useful when — Tasks are broad/decomposable, agents need different toolsets, one agent reviews another, workflows are long and specialized

Overkill when — Task is simple, a single orchestrated workflow can do it, complexity isn't justified

Q21. Supervisor pattern vs swarm pattern?

Supervisor — One orchestrator routes tasks to specialists. Central control. Good for clear domain separation.

Swarm — Agents communicate peer-to-peer, passing context between each other. Distributed. Good for emergent collaboration.

Q22. Stateless agent vs stateful agent?

Stateless — Each call is independent, no durable workflow state — simple but limited

Stateful — Keeps track of messages, decisions, tool outputs, progress — can pause/resume, supports long-running tasks

Stateful agents are more production-friendly for complex, multi-turn tasks.

Q23. What is a stop condition in an agent?

Defines when the agent should stop iterating:

- Final answer produced
- Enough evidence collected
- Max iterations reached
- Low confidence / escalation triggered
- Human approval required

Without stop conditions, agents can loop forever, burning tokens and time.

Q24. Why do agent loops need max iterations?

- Prevent infinite loops
- Stop repeated tool calls
- Control runaway cost
- Avoid slow responses
- Handle broken reasoning cycles

```
# LangGraph example
graph = graph.compile(
    checkpoint=checkpoint,
    recursion_limit=10 # max iterations
)
```

Q25. What are common failure modes of agents?

- Wrong tool selection — model picks irrelevant tool
- Bad tool arguments — model hallucinates parameters
- Looping — agent stuck repeating the same tool call
- Poor grounding — answers from model memory not retrieved context
- Using stale context — outdated documents
- Prompt injection from retrieved docs
- Taking actions without proper approval

Q26. How do you make agents more reliable?

- Good, unambiguous tool descriptions
- Explicit structured state schema
- Retries with limits and backoff
- Validation of tool arguments before execution
- Grounded retrieval (RAG) to reduce hallucination
- Human approval for risky actions
- Full observability / tracing

Q27. How do you debug an agent calling the wrong tool?

Inspect in this order:

1. System prompt / instructions
2. Tool descriptions — are they clear and non-overlapping?
3. Tool schemas — are parameter names obvious?
4. Conversation context — what context was the model given?
5. Retrieval context — did a retrieved doc confuse the model?
6. Traces of model reasoning via LangSmith

Common fix: improve tool descriptions, reduce tool overlap, add a router layer before the agent.

Q28. What is agent orchestration?

The logic that controls how the agent runs:

- What state is stored and how
- Which node/tool runs next
- Retries and error handling
- Human approval gates
- Stop conditions
- Checkpointing for resume

LangGraph is a dedicated framework for this.

Q29. How do agents and RAG work together?

RAG is often one **tool or sub-workflow inside an agent** — not a separate alternative.

Example: refund question flow

1. User: 'Can I return my order?'
2. Agent looks up order (order tool)
3. Agent retrieves refund policy docs (RAG tool)
4. Agent combines order data + policy
5. Agent decides response or next action

Q30. Pure RAG system vs agent with retrieval?

Pure RAG — Retrieve docs → put in prompt → answer. Fixed one-shot flow.

Agent with retrieval — Decides whether retrieval is needed, may retrieve multiple times, combines with other tools, routes based on evidence, may take action after retrieval

3. Advanced — Production Agent Systems

Q31. How would you design a production-grade AI agent system?

API Layer — FastAPI / backend service — auth, rate limits, request validation

Agent Runtime — LangGraph — state management, checkpointing, retries, conditional routing

Tool Layer — Internal APIs, retrieval tools, DB tools, external integrations (MCP servers)

Knowledge Layer — Vector DB / search, document store, metadata filters, reranking

Worker Layer — Background jobs for ingestion, long-running tasks (Celery / RQ)

Observability — Traces, tool call logs, token/cost metrics, failure monitoring (LangSmith)

Safety Layer — RBAC, approval gates, argument validation, action policies

Q32. Agent vs deterministic workflow — how do you decide?

Ask: Does the model need to make dynamic decisions?

Use deterministic workflow — Steps are known, logic is stable, correctness matters more than flexibility

Use agent — Tool choice depends on context, tasks vary, planning/reasoning needed, next step cannot be hardcoded

Q33. What is the biggest mistake people make with agentic systems?

Using an agent for everything. Many problems are better solved by a simple chain, a router + fixed workflow, a RAG pipeline, or a backend service with one tool call. If the task is deterministic, don't add agent complexity.

Q34. How would you handle memory in a production agent?

Separate into three purposeful layers:

Execution state (Current workflow) — Tool outputs, messages, current plan, retry counters — in-memory or Redis

Conversation memory (Current session) — Recent chat context — trimmed to last N turns or summarised

Durable memory (Across sessions) — User preferences, task summaries — only stable, useful information

Memory should be structured and purposeful — not one giant blob of text.

Q35. How do you prevent an agent from taking unsafe actions?

Never rely only on system prompts. Use layered controls:

1. Tool allowlist — agent can only call permitted tools
2. Server-side authorization (RBAC)
3. Input validation before tool execution
4. Confirmation step for risky/high-value actions
5. Human approval for destructive actions (LangGraph HITL)
6. Audit logs for every tool call
7. Rate limits and scope limits per user role
8. Restricted tool exposure based on auth context

Q36. What is human-in-the-loop (HITL) and when do you use it?

Agent pauses and waits for human approval or input.

- Refunds or compensation above a threshold

- Deleting records or bulk updates
- Sending legal / financial communication
- Low-confidence classification
- Compliance-required review

```
from langgraph.types import interrupt
```

```
def approval_node(state):
    if state['refund_amount'] > 500:
        # pause graph and wait for human
        human_decision = interrupt({'action': 'approve_refund', 'amount': state['refund_amount']})
        state['approved'] = human_decision['approved']
    return state
```

Q37. How would you evaluate an AI agent?

A) Task success

- Did it solve the user's task?
- Correct final answer?
- Correct action taken?

B) Tool behavior

- Did it pick the right tool?
- Correct arguments?
- Unnecessary or repeated tool calls?
- Loops / retries?

C) System metrics

- Latency p50/p95
- Cost per task
- Failure rate
- Escalation rate
- Human approval frequency

Q38. Agent observability vs normal API logging?

Normal logging: request/response level. Agent observability must also capture:

- Prompts used at each step
- Tool calls made and arguments
- Tool outputs / observations
- State transitions
- Retries and routing decisions
- Token usage per step
- Final answer quality

Without this, agent debugging is nearly impossible. Use LangSmith.

Q39. How do you reduce latency in an agent system?

- Use router before agent — avoid unnecessary agent reasoning for simple requests
- Reduce tool count exposed — fewer choices = faster decision
- Cache retrieval / tool results for repeated queries
- Parallelize independent tool calls
- Use smaller model for classification / routing

- Limit max iteration count
- Stream final response while generating

Q40. How do you reduce cost in an agent system?

- Keep prompts smaller — trim message history
- Avoid repeated retrieval of same docs — cache embeddings
- Use small model for routing, large model only when needed
- Reduce failed tool loops with better tool descriptions
- Cache common tool outputs (Redis TTL)
- Use deterministic code instead of LLM where possible

Q41. How do you protect against prompt injection in agents?

Critical for agents that process user content or retrieved docs:

- Treat retrieved/web content as untrusted — user-level trust, not system-level
- Keep system instructions structurally separate from retrieved text
- Never allow retrieved docs to redefine agent behavior
- Restrict sensitive tools — don't expose destructive tools to untrusted input paths
- Validate actions server-side regardless of model output
- Add approval gates for high-risk actions

Q42. How do you test an agentic system end-to-end?

- Unit test each LangGraph node in isolation with mock state
- Integration test with mock LLM + mock tools
- End-to-end test with real LLM against a labeled eval dataset
- Use LangSmith datasets for regression testing on prompt changes
- Test HITL interrupt/resume paths explicitly
- Test failure paths: tool errors, retry exhaustion, escalation triggers

Q43. What are signs of a badly designed agent architecture?

- Too many overlapping tools — model can't decide which to use
- No clear stop conditions — loops forever
- No observability — can't debug what happened
- No approval flow for risky actions
- Giant unstructured memory blobs
- Using an agent where a simple service would work
- Agent repeatedly calling the same tool
- Prompts doing business logic that should be in code

Q44. How would you design a safe agent for high-value actions?

Layered safety architecture

- | | |
|--------------------|----------------------------------|
| 1. System prompt | → tool usage rules |
| 2. Tool allowlist | → only permitted tools |
| 3. RBAC | → server-side auth per tool |
| 4. Arg validation | → validate before execution |
| 5. Threshold check | → flag high-value actions |
| 6. HITL interrupt | → pause for human if > threshold |
| 7. Audit log | → every tool call logged |

8. Idempotency → safe to retry without side effects

Q45. How do you decide: non-agentic vs single-agent vs multi-agent?

Non-agentic workflow — Steps are known, stable, high determinism needed — use chains/pipelines

Single agent — Multiple tools, dynamic decisions needed, one reasoning loop is enough

Multi-agent — Tasks naturally decomposable, separate specialists improve quality, complexity is justified

Q46. What is the role of checkpointing in production agents?

Persists workflow state so execution can:

- Resume after failure or process restart
- Pause for human approval
- Support long-running workflows (minutes/hours)
- Debug by replaying from a known state

```
from langgraph.checkpoint.redis import RedisSaver

checkpointer = RedisSaver.from_conn_string('redis://localhost:6379')
graph = workflow.compile(checkpointer=checkpointer)

# Resume from thread_id
graph.invoke(input, config={'configurable': {'thread_id': 'user-123'}})
```

Q47. How would you handle tool failures and retries in LangGraph?

```
def tool_node(state):
    try:
        result = call_external_tool(state['args'])
        return {'**state', 'tool_result': result, 'tool_failed': False}
    except Exception as e:
        return {'**state', 'tool_failed': True, 'error': str(e)}

def route_after_tool(state):
    if state['tool_failed'] and state['retry_count'] < 3:
        return 'retry_node'
    elif state['tool_failed']:
        return 'escalation_node'
    return 'continue_node'

graph.add_conditional_edges('tool_node', route_after_tool)
```

Q48. How do you handle LLM context window limits in long agent workflows?

- Trim message history — keep last N turns only
- Summarise older messages instead of dropping
- Store tool outputs in state, inject only relevant ones per step
- Use external memory (DB/Redis) and retrieve on demand
- Avoid passing entire documents — chunk and retrieve

Q49. How do you ensure determinism in a production agent?

- Set temperature=0 for classification/routing steps
- Use structured output / JSON mode for tool selection
- Validate model output before executing any tool

- Use explicit conditional edges instead of relying on model judgment for routing
- Log and test every conditional path

Q50. Explain agentic AI to an interviewer in one answer.

"Agentic AI means building systems where an LLM is not limited to a single text response, but can operate in a goal-directed loop: understand the task, decide what action or tool is needed, observe results, update state, and continue until the task is completed or a stop condition is met. In practice, a production agent needs more than an LLM — it needs orchestration, tool integration, memory/state, safety controls, observability, and often retrieval and human approval flows."

4. Agent vs Agentic — Key Distinctions

Q51. What is the difference between 'agent' and 'agentic'?

Agent — A concrete system/component that can perform actions toward a goal

Agentic — A style/behavior/pattern — planning, autonomy, tool use, iterative execution, decision-making across steps

Agent = the thing. Agentic = the behavior/pattern.

Q52. Is every tool-calling LLM an agent?

Not necessarily. If it calls one tool in a fixed way, that's a tool-augmented LLM workflow. It becomes agent-like when it can: choose among tools, decide next steps dynamically, maintain state across steps, and iterate toward a goal.

Q53. Is every RAG app agentic?

No. Basic RAG is: retrieve → generate. That's not agentic. It becomes agentic if the system dynamically decides: whether to retrieve, what to retrieve, whether to call another tool, whether to ask follow-ups, or whether to escalate.

Q54. Is an agent always better than a workflow?

No. A workflow is often better when: business logic is known, you need predictability, correctness matters more than flexibility, latency and cost matter. Agents are powerful, but not automatically better.

Q55. What is the best practical definition of agentic AI for interviews?

"Agentic AI is an approach where the model is given a goal and can iteratively decide actions, use tools, observe results, and update its next step instead of producing only a one-shot response. The 'agentic' part is the dynamic decision-making loop, not just the presence of an LLM."

5. Architecture Deep Dives

E-commerce Support Agent (your Star Gallery project)

Tools:

- `order_lookup(order_id)`
- `shipment_tracking(order_id)`
- `refund_eligibility(order_id)`
- `product_lookup(product_id)`
- `jira_create_ticket(issue_details)`

Knowledge / RAG:

- Refund policy
- Shipping policy
- FAQ
- Product care instructions

Workflow (LangGraph):

```
classify_intent
  ■■■ order_related → fetch_order → combine_with_policy → answer
  ■■■ policy_related → retrieve_docs → answer
  ■■■ issue_detected → create_jira_ticket (tool use)
  ■■■ complex/unclear → human_escalation
```

Safety: no refunds without auth + policy checks, HITL for large compensation.

Autonomous SRE Agent (your Star Gallery project)

Tools:

- `check_ssl_expiry(domain)`
- `renew_ssl(domain)`
- `check_aws_health(service)`
- `check_domain_expiry(domain)`
- `send_alert(message)`

Workflow:

```
scheduled_trigger
  → run_health_checks (parallel: SSL, domain, AWS)
  → evaluate_results
    ■■■ all_healthy → log_ok
    ■■■ ssl_expiring_soon → renew_ssl
    ■■■ aws_issue → send_alert + create_incident
    ■■■ critical → page_oncall (HITL escalation)
```

Internal Enterprise Assistant

- FastAPI backend + SSO / RBAC
- LangGraph workflow for orchestration
- MCP servers: Confluence, Jira, GitHub, Slack, internal DBs

- RAG for internal documentation
- Supervisor agent routing to specialist sub-agents

classify_request

- knowledge_question → confluence_search + answer
- ticket_query → jira_lookup + summarize
- code_question → github_search + explain
- action_request → auth_check → execute + audit_log

6. Top 20 Must-Memorise Questions

Highest ROI — master these first:

Foundations

- What is an AI agent?
- What is agentic AI?
- LLM app vs agent — key difference
- Chain vs agent — when to use each
- What is a tool in an agent?

Core Mechanics

- What is ReAct (Reason → Act → Observe)?
- What is agent state?
- Short-term vs long-term memory
- What is planning — when is it needed?
- Tool calling vs agentic behavior

Reliability & Safety

- Common failure modes of agents
- How to prevent infinite loops
- How to debug wrong tool selection
- How to prevent unsafe actions (layered safety)
- Prompt injection in agent systems

Production Design

- When to use agent vs deterministic workflow
- How to design a production-grade agent system
- How to evaluate an agent (3 levels)
- Single-agent vs multi-agent — decision rule
- Agent + RAG relationship

7. What Interviewers Ask After Agent Questions

Once agent basics are covered, interviews move to architecture + build questions:

- Design a customer support agent for e-commerce
- Design a multi-tool AI assistant for internal docs + DB + ticketing
- Build an order tracking + refund assistant — what tools, what flow?
- How do you prevent an agent from issuing unsafe refunds?
- How do you evaluate whether the agent is actually helping users?
- How would you store memory / state across long conversations?
- LangChain agent vs LangGraph workflow — when do you choose each?
- Where does MCP fit in an enterprise agent stack?
- How do you reduce agent cost and latency at scale?
- How do you debug an agent loop that never terminates?

8. Interview Checklists

Agent Concepts Checklist

- AI agent definition
- Agentic AI definition
- Agent vs chain
- ReAct loop
- Agent state
- Short-term vs long-term memory
- Planning — when to use
- Stop conditions
- Max iterations
- Tool calling vs agentic behavior

Production Agent Checklist

- Layered safety (allowlist → RBAC → validation → HITL → audit)
- Checkpointing for resume/pause
- Human-in-the-loop for high-risk actions
- Observability (LangSmith traces)
- 3-level evaluation (task / tool behavior / system metrics)
- Prompt injection defence
- Retry handling with limits
- Multi-agent: supervisor vs swarm pattern
- Context window management in long workflows
- Cost reduction: routing, caching, small model first

Architecture Patterns Checklist

- Router agent pattern
- Supervisor agent pattern
- Swarm pattern
- RAG as a tool inside an agent
- HITL interrupt/resume in LangGraph
- Tool failure + retry flow
- Stateful vs stateless agent
- Non-agentic vs single-agent vs multi-agent decision rule

For your profile: be ready to walk through your Agentic RAG Support Assistant and SRE Agent architectures in detail — why LangGraph, how tool use was scoped, how the Jira ticket creation was triggered safely, what state was carried, and how you'd add checkpointing and HITL to make them fully production-grade.